

【문제 A】 문제 1 유형 — 시프트 레지스터 패턴 감지

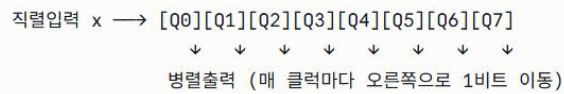
74164 시프트 레지스터를 이용해서, 최근 5개의 입력 중 1이 홀수 개(1개 또는 3개 또는 5개) 입력 되었을 때 출력 $z = 1$ 이 되는 시스템을 설계하라.

- 74164는 8비트 직렬입력-병렬출력(SIPO) 시프트 레지스터이다
- 조합논리 게이트를 추가로 사용할 수 있다
- 블록도와 출력 논리식을 함께 제시하라

【문제 A】 이동 레지스터 홀수 개 1 감지

Step 1 — 74164 구조 이해

74164는 8비트 직렬입력-병렬출력(SIPO) 이동 레지스터:



최근 5개 입력만 필요하므로 $Q_0 \sim Q_4$ 5비트만 사용

Step 2 — 홀짝 검사 원리

"1의 개수가 홀수" = 홀짝 검사(Parity Check) = 1

홀짝 검사는 XOR(배타적 논리합)으로 구현:

$$z = Q_0 \oplus Q_1 \oplus Q_2 \oplus Q_3 \oplus Q_4$$

원리:

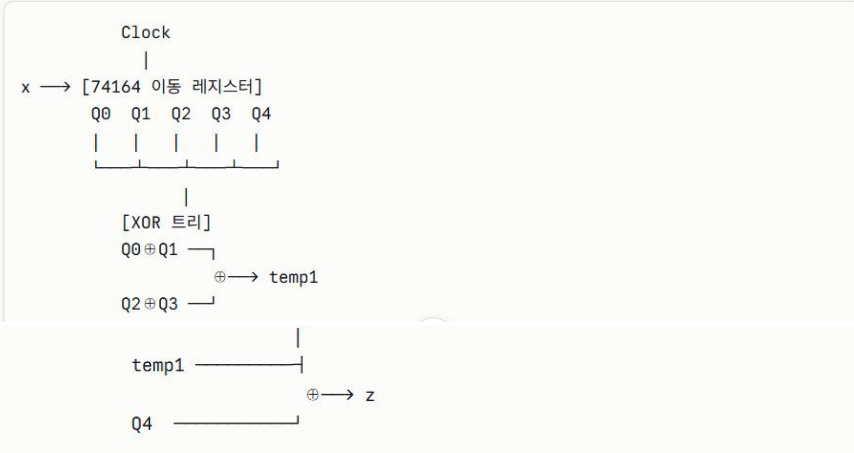
- 짝수 개의 1 $\rightarrow$ XOR 결과 = 0
- 홀수 개의 1 $\rightarrow$ XOR 결과 = 1

Step 3 — 동작 검증

Q0	Q1	Q2	Q3	Q4	1의 개수	z
1	0	0	0	0	1(홀수)	1
1	1	0	0	0	2(짝수)	0
1	1	1	0	0	3(홀수)	1

1	1	1	1	0	4 (짝수)	0
1	1	1	1	1	5 (홀수)	1

Step 4 — 블록도



Step 5 — 최종 논리식

$$z = Q_0 \oplus Q_1 \oplus Q_2 \oplus Q_3 \oplus Q_4$$

필요 소자: 74164 × 1 + XOR 게이트 × 4

【문제 B】 문제 4 유형 — 카운터 / 시프트 레지스터 조건 출력

입력이 정확히 5 클럭 동안 연속으로 1이면 출력이 1이 되는 시스템을 설계하라. 조합논리 블록 이외에도 다음과 같은 것들을 이용할 수 있다.

- a. 하나의 4비트 카운터
- b. 하나의 8비트 시프트 레지스터

각각의 방법에 대해 블록도와 동작 설명을 제시하라.

【문제 B】 연속 5클럭 입력=1 감지

방법 a — 4비트 카운터 사용

Step 1 — 설계 아이디어

- x=1이면 카운터 증가
- x=0이면 카운터 즉시 리셋(CLR)
- 카운터가 5(0101)에 도달하면 z=1 출력

상태 변화:

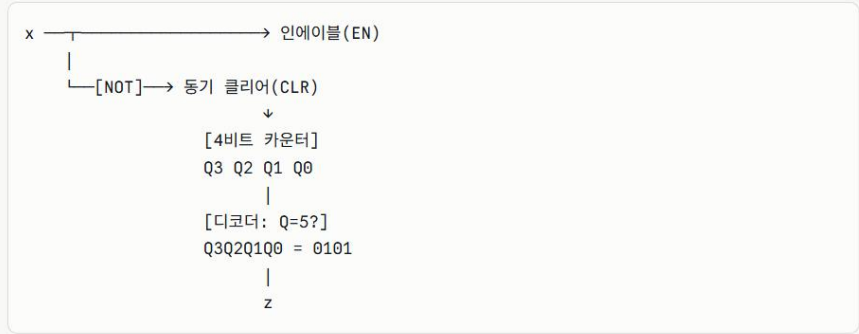
x 연속 1: 0→1→2→3→4→5 → z=1

x=0 등장: 즉시→0 (리셋)

Step 2 — 카운터 연결 방식



Step 2 — 논리식



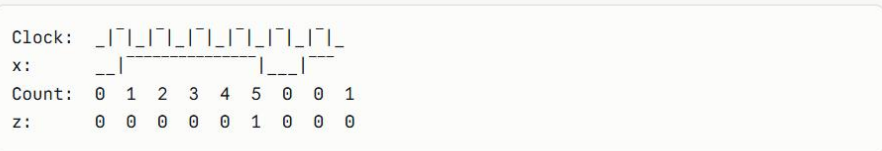
Step 3 — 출력 논리식

5 = 이진수 0101 이므로 Q3=0, Q2=1, Q1=0, Q0=1:

5 = 이진수 0101 이므로 Q3=0, Q2=1, Q1=0, Q0=1:

$$z = \overline{Q_3} \cdot Q_2 \cdot \overline{Q_1} \cdot Q_0$$

Step 4 — 타이밍 동작



방법 b — 8비트 이동 레지스터 사용

Step 1 — 설계 아이디어

- 입력 x를 매 클럭마다 이동 레지스터에 저장
- 최근 5개 비트(Q0~Q4)가 모두 1이면 z=1
- 5입력 AND 게이트 하나로 구현 가능



Step 2 — 출력 논리식

$$z = Q_0 \cdot Q_1 \cdot Q_2 \cdot Q_3 \cdot Q_4$$

Step 3 — 타이밍 동작

Clock: 

x: 

Q0: 1 1 1 1 1 0 0 1

Q1: 1 1 1 1 1 0 0

Q2: 1 1 1 1 1 0

Q3: 1 1 1 1 1

Q4: 1 1 1 1

z: 0 0 0 0 0 1 0 0

Step 4 — 두 방법 비교

항목	4비트 카운터	8비트 이동 레지스터
필요 소자	카운터+디코더+클리어 논리	이동 레지스터+AND

회로 복잡도	보통	간단
확장성	N 변경 시 디코더만 수정	N 변경 시 AND 입력 수 변경
직관성	보통	높음

【문제 C】 문제 11 유형 — 최근 입력 조건 블록도

최근 4개의 입력 중에서 정확히 2개(현재 값 포함)가 1이면 출력 $z=1$ 이 되는 시스템의 블록도를 보
여라. 상태표나 상태도는 보일 필요가 없고, 어떤 종류의 플립플롭과 게이트도 사용할 수 있다.

【문제 C】 수정판 — 최근 4개 입력 중 정확히 2개가 1

Step 1 — 최근 4개 입력 저장

D 플립플롭 4개로 이동 레지스터 구성:

x → [D0] → [D1] → [D2] → [D3]

Q0 Q1 Q2 Q3

(현재) (1클럭전) (2클럭전) (3클럭전)

Step 2 — "정확히 2개가 1" 논리

4비트 중 정확히 2개가 1인 경우 $C(4,2) = 6$ 가지:

경우 Q3 Q2 Q1 Q0

1	0	0	1	1
2	0	1	0	1
3	0	1	1	0
4	1	0	0	1
5	1	0	1	0
6	1	1	0	0

Step 3 — 방법①: 6개 최소항 직접 검출

$$z = \overline{Q_3} \overline{Q_2} Q_1 Q_0 + \overline{Q_3} Q_2 \overline{Q_1} Q_0 + \overline{Q_3} Q_2 Q_1 \overline{Q_0} + \overline{Q_3} \overline{Q_2} \overline{Q_1} Q_0 + \overline{Q_3} \overline{Q_2} Q_1 Q_0 + \overline{Q_3} Q_2 \overline{Q_1} \overline{Q_0}$$

Step 4 — 방법②: 3비트 가산기 (고급 설계)

4개 입력을 진짜 3비트로 합산:

$$N = Q_0 + Q_1 + Q_2 + Q_3 = (C_2 C_1 C_0)_2$$

회로 구성:

1단계: 반가산기 HA1

입력: Q0, Q1

출력: s_a = Q0 ⊕ Q1, c_a = Q0 · Q1

2단계: 반가산기 HA2

입력: Q2, Q3

출력: s_b = Q2 ⊕ Q3, c_b = Q2 · Q3

3단계: 전가산기 FA

입력: s_a, s_b, c_a

출력: C0 = s_a ⊕ s_b ⊕ c_a

C1 = (s_a · s_b) + (s_a · c_a) + (s_b · c_a)

4단계: 올림수 처리

C2 = c_b OR (FA의 올림수)

N=2 (010) 감지:

$$z = \overline{C_2} \cdot C_1 \cdot \overline{C_0}$$

Step 5 — 반례 검증

입력	1의 개수	방법①	방법②	올바른 z
1111	4	최소항 없음 → 0 ☒	N=100 → C2=1 → 0 ☒	0
1010	2	경우5 해당 → 1 ☒	N=010 → C1=1 → 1 ☒	1
0000	0	최소항 없음 → 0 ☒	N=000 → 0 ☒	0
1100	2	경우6 해당 → 1 ☒	N=010 → 1 ☒	1

Step 6 — 두 방법 비교

방법	장점	단점
----	----	----

① 6개 최소항	단순 직관적	AND 게이트 6개 + OR 게이트 1개 필요
② 3비트 가산기	확장성 높음, 회로 재사용 가능	설계 복잡도 높음